PS 2 — Motion Planning (Part A)

The main idea of PS 2 is this: turn the world into a grid of squares, then search the grid step by step for a route from start to goal.

Sources: Red Blob Games, "Introduction to the A* Algorithm" and "Implementation of A*" (redblobgames.com); Gabriel Gambetta, "Pathfinding Demystified, Part I" (gabrielgambetta.com). The Red Blob pages are interactive. Drag the start and goal around and watch each algorithm explore, because it will make everything below click.

1. Path planning fundamentals

Difference between a path and a trajectory.

- A **path** is only *where* to go: an ordered list of positions, like a line drawn on a map.
- A **trajectory** is a path plus *when*: positions with time attached, which also fixes speed and acceleration at each point. For example, "be at point A at 0 s, point B at 2 s, moving at 3 m/s."
- Two cars can follow the same path with different trajectories, one slowly and one quickly.

Waypoint, obstacle, free space, start state and goal.

Term	Meaning	Everyday example
Waypoint	An intermediate point the vehicle should pass through	Stops on a delivery route
Obstacle	Anything the vehicle must not enter	A wall, parked car, or person
Free space	All the places the vehicle can safely be	The open road
Start state	Where the vehicle is now, and possibly its heading and speed	Your house
Goal	Where it needs to reach	The college gate

2. Environment representation

What is a 2D grid map? You divide the area seen from above into equal squares, like a chessboard. Each square, called a cell, represents a small patch of ground, for example 10 cm × 10 cm. In code it is a 2D array, a table of numbers, where grid[row][col]

describes one cell. Moving from one cell to a neighbouring cell is one step. Neighbours are up, down, left and right (4-connected), plus the diagonals if allowed (8-connected).

What is an occupancy grid? It is a grid map where each cell stores whether it is occupied by an obstacle. In real robots it stores a *probability* of being occupied, such as 0.9, built up from sensor readings like LiDAR. For PS 2 you can keep it simple, with 0 meaning free and 1 meaning obstacle.

Free, occupied and unknown cells.

- **Free**: sensors have seen it and it is empty, so the vehicle may drive there.
- **Occupied**: something is there, so it must be avoided.
- **Unknown**: the sensors have not seen it yet, perhaps because it is behind a wall or out of range. Planners usually treat unknown cells carefully, either avoiding them or giving them a higher cost.

3. Search algorithms

All three algorithms share the same skeleton. They keep a **frontier**, the list of cells waiting to be explored. Repeatedly they take one cell out, look at its neighbours, and add new ones to the frontier. They record **where each cell was reached from** (came_from). When the goal is reached, they walk came_from backwards from the goal to the start to get the path. They differ only in **which cell they take out next**.

BFS (Breadth-First Search): working principle and intuition.

- The frontier is a simple queue: first in, first out.
- It explores in rings spreading out from the start, like ripples from a stone dropped in water. First every cell 1 step away, then every cell 2 steps away, and so on.
- Because it goes ring by ring, the first time it reaches the goal it has found a path with the **fewest steps**.
- Drawback: it explores in every direction equally, even away from the goal, so it visits many cells. It also ignores costs, treating every step as equal.

Dijkstra's algorithm: working principle and how it differs from BFS when costs are not equal.

- Sometimes steps cost different amounts. Mud or grass might cost 5 and road might cost 1.
- Dijkstra keeps track of the **total cost so far** from the start to each cell, called g. It always expands the frontier cell with the **lowest total cost**, using a priority queue (a list that always hands you the smallest item first).

- If a cheaper way to reach a cell is found later, it updates that cell's cost.
- Result: it finds the **cheapest** path, not just the shortest in steps. With equal costs it behaves like BFS. With unequal costs, BFS might go straight through the mud, while Dijkstra goes around it if that is cheaper overall.

A* algorithm: working principle and the role of the heuristic.

- A* is Dijkstra plus a sense of direction.
- For each cell it computes f = g + h:
    - **g**: the real cost from the start to this cell, as in Dijkstra.
    - **h**: the **heuristic**, an *estimate* of the remaining cost from this cell to the goal. On a grid the usual choice is the Manhattan distance, |x1 − x2| + |y1 − y2|, the number of steps if there were no walls.
- It always expands the cell with the smallest f, so it prefers cells that are both cheap to reach and look close to the goal. It heads toward the goal instead of spreading in all directions.
- **Role of the heuristic:** it guides the search and greatly reduces the cells explored. If h never *overestimates* the true remaining cost (it is "admissible"), A* still guarantees the cheapest path. If h = 0, A* becomes Dijkstra.

Algorithm	**Picks next**	**Handles unequal costs?**	**Finds best path?**	**Cells explored**
BFS	Oldest cell in queue	No	Fewest steps	Many
Dijkstra	Lowest cost so far (g)	Yes	Cheapest	Many
A*	Lowest g + h	Yes	Cheapest, if h is admissible	Fewest

4. Global and local planning

Difference between a global path and a local motion decision.

- A **global path** is the overall route from start to goal, computed on the whole map, like a Google Maps route through the city. It is computed rarely and does not react quickly.
- A **local motion decision** is what to do in the next second or two: how much to steer and how fast to go, to follow the global path while avoiding nearby things

the map did not show, such as a pedestrian stepping out. It runs many times per second using live sensor data.

Why re-plan when a new obstacle appears? The global path was computed with the map as it was known at that moment. If a new obstacle now blocks the path, such as a fallen tree or a parked lorry, following the old path leads to a collision or a dead end. The vehicle marks those cells as occupied, then runs the planner again from its *current* position to find a new route. Real environments change constantly, so planning is repeated rather than done once.